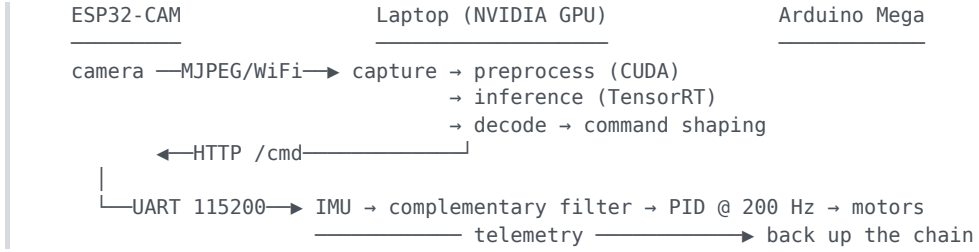
REFLEX-CV

# Code Reading Guide & Field Manual

How to read a vision-guided self-balancing robot's codebase in the right order, and how to bring it up on real hardware without breaking it.




**Repository** awabasif1/gyrobot · **~7,000 lines** of C++/CUDA/Arduino across 26 source files
**Companion documents** BUILD_GUIDE.md (project plan & hardware) · mcu/SERIAL_PROTOCOL.md (the board-to-board contract)

CONTENTS

# What is in this document

Part 1 is for sitting at a desk with the repository open. Part 2 is for standing next to the robot. Part 3 says exactly which parts of this codebase have been proven to work and which have not — read it before you trust anything.


| | | |
|---|---|---|
| 0 | **The one idea behind the architecture** | Why three processors, and why that is not over-engineering |
| 1 | **How to read this codebase** | A reading order, a file map, the seven decisions worth understanding, and where the subtle code hides |
| 2 | **Bringing it up in the real world** | Seven gated stages from laptop to balancing robot, with a safety rule set and a tuning procedure |
| 3 | **Tuning, measuring, and what breaks** | PID and tracking gains, filling in the latency table, and a symptom-to-cause table |
| 4 | **Verification status — what is actually proven** | Compiled, tested, type-checked only, or never built at all. Per file. |
| A | **Appendix: command-line reference** | |
| B | **Appendix: every constant you will want to change** | |


## If you only have thirty minutes

Read these four things, in this order, and you will understand the system well enough to work on either half of it:

1. `mcu/SERIAL_PROTOCOL.md` — the contract between the boards. Everything else is an implementation of it.
2. Part 0 of this document — one page, the reason the system is shaped this way.
3. `src/main.cpp`, the `while` loop only (about 120 lines) — the whole vision runtime in dataflow order.
4. `mcu/balance_controller.ino`, the `loop()` function only (about 100 lines) — the whole control runtime, in eight numbered steps.

PART 0

# The one idea behind the architecture

Almost every structural decision in this repository follows from a single observation: **perception is slow and unpredictable, and balancing cannot tolerate unpredictability at all.**

A camera frame takes 20 ms to arrive on a good day and 200 ms on a bad one, because it crosses a WiFi link shared with every other laptop in the room. GPU inference takes 8 ms most frames and 25 ms when the driver decides to do something else. None of that is fixable; it is what networked perception costs.

Meanwhile, an inverted pendulum with its centre of mass 12 cm above the axle falls over on a timescale of roughly `sqrt(L/g)` ≈ 110 ms. A control loop that occasionally pauses for 200 ms to wait for a JPEG does not balance a robot. It drops it.

So the two jobs are put on different processors, and the slow one is never allowed to block the fast one:

| Processor | Rate | Job, and what happens when it stalls |
|---|---|---|
| **Arduino Mega** | 200 Hz, fixed | Read the IMU, run the PID, drive the motors. Never waits for anything. If no command has arrived for 400 ms it zeroes the steering and keeps standing up. |
| **Laptop + GPU** | 15–30 Hz, variable | Find the target, decide where to go. If it crashes, the robot stops steering and stands still. |
| **ESP32-CAM** | best effort | A camera with an antenna and a serial port. Knows nothing about balancing. If it dies, same as above. |

> **THE INVARIANT TO PROTECT**
>
> The balance loop reads whatever command arrived *most recently* and runs regardless. It never blocks, never retries, never waits. Any change that introduces a wait into `loop()` in `balance_controller.ino` — a `delay()`, a blocking serial read, a library call with internal retries, a `Serial.print` of a float in the control path — breaks the one property that makes this system work.

This also explains a structural choice that looks like extra work: the degradation path is layered three deep, and each layer works when the others are gone.

| Layer | Where | What it covers |
|---|---|---|
| 1. Target lost | Laptop, `CommandShaper` | Hold the last command 350 ms, then ramp to zero over 400 ms. Covers a missed detection or one corrupt frame. |
| 2. Command timeout | Mega, `CMD_TIMEOUT_MS` | No valid line for 400 ms → steering forced to zero. Covers the laptop crashing, WiFi dropping, the UART cable falling out. |

| | | |
|---|---|---|
| 3. Clean shutdown | Laptop, Ctrl-C handler | Sends `L:0.00,T:0.00` before exiting. Without it, quitting the program leaves the Mega turning forever. |

Layer 2 does not depend on layer 1, and layer 3 depends on neither. The Mega's timeout is the authoritative one, because the Mega is the only board that still functions when the other two are gone.

PART 1

# How to read this codebase

## 1.1 The reading order

Read in dataflow order, not alphabetical order, and read the contract before any implementation of it. Times are for reading with understanding, not skimming.

| # | File | Lines | Time | Why here |
|---|---|---|---|---|
| 1 | `mcu/SERIAL_PROTOCOL.md` | 194 | 15 min | The contract all three boards implement. Read this first or nothing else has context. |
| 2 | `src/preprocess_math.cuh` | 184 | 20 min | The letterbox geometry. Two separate stages depend on it agreeing with itself; understand it before either. |
| 3 | `src/preprocess.cu` | 236 | 40 min | The custom CUDA kernel. The densest file in the repo and the one with the most comments explaining *why*. |
| 4 | `src/trt_infer.h / .cpp` | 413 | 25 min | Engine loading and buffer ownership. Mostly careful error handling; skim the constructor's checks. |
| 5 | `src/postprocess.h / .cpp` | 259 | 25 min | Decoding 8,400 candidate boxes into one target. Plain C++, no GPU. |
| 6 | `src/command_shaper.h / .cpp` | 233 | 20 min | Smallest file, largest influence on whether the robot looks controlled. Read every line. |
| 7 | `src/esp32_client.h / .cpp` | 562 | 25 min | Read the header comment, then the worker thread. The socket code below it is boilerplate. |
| 8 | `src/main.cpp` | 726 | 45 min | Ties it together. Read `FrameGrabber`, then the `while` loop. Skip argument parsing and the overlay. |
| 9 | `mcu/balance_core.h` | 474 | 45 min | All the firmware logic, Arduino-free. Filter, PID, parser, safety, mixing. |
| 10 | `mcu/balance_controller.ino` | 618 | 40 min | The hardware layer plus the tuning procedure at the bottom. |
| 11 | `esp32cam/…uart_bridge.ino` | 604 | 25 min | Read the header comment and the three endpoint handlers. |
| 12 | `test/*.cpp` | 1,609 | 30 min | Read these as *specifications*. Each assertion is a claim about intended behaviour. |

> **A SHORTCUT THAT WORKS**
>
> The four test files are the fastest way to understand what the code is *supposed* to do, because every assertion is an executable statement of intent. `test_balance_core.cpp` in particular ends with a closed-loop inverted pendulum simulation — reading it tells you more about how the control loop is meant to behave than reading the controller does.

## 1.2 File map: who owns what

| File | Owns |
|---|---|
| **`src/`** | |
| `preprocess_math.cuh` | Letterbox geometry, bilinear weights, shared-memory tile sizing. Compiles on host *and* device. |
| `preprocess.cu / .cuh` | Two CUDA kernels (shared-memory tile and global fallback) plus the launch planner that picks between them. |
| `trt_infer.cpp / .h` | One deserialised TensorRT engine, its device buffers, and all the "your engine is wrong" error messages. |
| `postprocess.cpp / .h` | YOLOv8 output decode, IoU, NMS, sticky target selection. No CUDA, no TensorRT. |
| `command_shaper.cpp / .h` | Detection → lean/turn, with deadband, low-pass, slew limit, and target-loss decay. |
| `esp32_client.cpp / .h` | The command link: a worker thread, a one-slot mailbox, a keep-alive socket. |
| `main.cpp` | Capture thread, pipeline orchestration, profiling, overlay, shutdown. |
| **`include/`** | |
| `cuda_utils.cuh` | `CUDA_CHECK`. Every CUDA call in the project goes through it. |
| `stage_profiler.cuh` | CUDA-event timing, for GPU stages only. |
| `host_profiler.h` | `steady_clock` timing, for the CPU and network stages CUDA events cannot measure. |
| **`mcu/`** | |
| `balance_core.h` | Complementary filter, PID with anti-windup, line parser, safety state machine, motor mixing, encoder velocity. |
| `balance_controller.ino` | I2C IMU reads, PWM, UART, the 200 Hz tick, and the tuning procedure. |
| `imu_calibrate.ino` | Gyro bias measurement and the angle-sign check. **Run before anything else.** |
| `uart_echo_test.ino` | Link test that cannot move the robot — motors are never touched. |
| **`esp32cam/`** | |
| `…uart_bridge.ino` | Camera init, MJPEG stream on :81, `/cmd` `/telemetry` `/status` `/res` on :80, UART relay. |
| **`tools/ and test/`** | |

| | |
|---|---|
| `verify_preprocess.cpp` | Compares the CUDA kernel against two OpenCV CPU references and benchmarks both kernel variants. |
| `test/*.cpp` | Four host suites. No GPU, no TensorRT, no OpenCV, no network. |
| `test/stubs/NvInfer.h` | A fake TensorRT header for type-checking only. **Not** part of the real build. |

## 1.3 Seven decisions worth understanding

These are the choices that will look arbitrary until you know the reason, and that you might otherwise "simplify" into a bug.

### ① The letterbox geometry lives in one shared header

`preprocess_math.cuh` is included by both the CUDA kernel and the CPU postprocessor. The kernel samples *along* a coordinate mapping; the postprocessor has to *invert* it to put boxes back on the camera frame. If those two disagree by half a pixel, every detection lands slightly off-centre, the turn command carries a constant bias, and the robot steers to one side of the target forever — while looking like it is merely badly tuned. So the mapping is defined once and both sides call it. The roundtrip is unit-tested: `src → dst → src` must be the identity.

### ② The kernel is fused, and there are two of them

Resize, BGR→RGB, `/255`, and HWC→CHW happen in one pass. Each is memory-bound alone, so four passes would move the frame through GPU memory four times to do a handful of arithmetic operations.

There are two kernels because the shared-memory optimisation has a limit. A 32×8 output tile needs a source window of roughly `(32/scale) × (8/scale)` pixels; as the camera resolution rises relative to the model input, that window grows until it will not fit in a block's shared memory. Rather than shrink the tile — which would break the coalesced 128-byte stores that make the fast path fast — the launcher falls back to a kernel that reads taps straight from global memory. Same arithmetic, same output, verified identical by `verify_preprocess`.

```
measured shared-memory requirement per block (32×8 tile)

  640×480  → 640×640    window 34×10     1,360 B     shared path
  800×600  → 640×640    window 42×12     2,016 B     shared path
 1024×768  → 640×640    window 54×15     3,240 B     shared path
 1600×1200 → 640×640    window 82×22     7,216 B     shared path
                                      ↑ limit is 48 KB; fallback is rarely hit
```

### ③ The block is 32×8, not 16×16

Both are 256 threads. 32 wide means one warp's 32 lanes write 32 consecutive floats — exactly 128 bytes, one full cache-line transaction — into the NCHW output. A 16-wide tile would split every warp's store across two output rows and two 64-byte segments, for no gain anywhere else. 256 threads is also the number that lets 6 blocks saturate an sm_86 SM's 1,536-thread limit.

### ④ The command link owns a thread and a one-slot mailbox

The naive version calls an HTTP GET inline, once per frame. When WiFi hiccups, that call stalls the entire vision loop, frames pile up, and the commands that eventually arrive describe where the target

used to be.

Instead, `post_command()` drops the newest command into a single-slot mailbox and returns immediately. A worker thread sends it. If the network is slower than the vision loop, intermediate commands are *overwritten and never sent* — which is correct, because a stale steering command has negative value. Overwrites are counted and displayed, so the behaviour is visible rather than silent.

#### ⑤ The firmware's logic contains no Arduino

`balance_core.h` has no `#include <Arduino.h>`, no `String`, no `Serial`, no `delay()`, and no dynamic allocation. Everything hard about the firmware — sensor fusion, PID anti-windup, the line parser, the arm/disarm state machine — is in there, and all of it is tested on a laptop at thousands of simulated timesteps per second. `balance_controller.ino` is the thin shell that reads the IMU and writes the motors.

This is not an abstraction for its own sake. On real hardware a bug in the parser, a bug in the PID, and a loose wire all present identically: the robot falls over and the only output is a serial cable.

#### ⑥ The GPU pipeline runs on the default stream, deliberately

The stages are strictly dependent — preprocess needs the upload, inference needs preprocess — so there is nothing to overlap. More importantly, `StageProfiler` records its CUDA events without naming a stream. If the work were enqueued on a non-default stream, those events would not bracket the work they claim to measure, and the latency table would be quietly wrong. One stream keeps the measurement honest.

#### ⑦ The ESP32 runs two HTTP servers

An MJPEG handler never returns; it loops pushing frames until the client disconnects. `esp_http_server` serves one request at a time per instance. Sharing a port would therefore queue `/cmd` behind the stream forever — the robot would receive video and no commands at all. Control is on port 80, the stream on port 81.

### 1.4 Where the subtle code is

Six places where the obvious version is wrong. Each is commented in place; they are collected here so you know they exist before you edit near them.

| Location | The trap |
|---|---|
| `balance_controller.ino` `wire_read16()` | The obvious `(Wire.read() << 8) \| Wire.read()` is **undefined-order** in C++ — the compiler may read the low byte first, producing byte-swapped IMU samples. Two separate statements are sequenced; one expression is not. |
| `balance_controller.ino` the PID call site | The PID output is **negated**. The textbook error is `setpoint − measurement`, which is negative for a forward lean, but a forward lean must be caught by driving the wheels *forward*. The closed-loop simulation in the test suite exists to pin this down; with the sign flipped it diverges. |
| `balance_core.h` `Pid::update()` | D acts on the *measurement*, not the error, so a setpoint step — which is exactly what an incoming vision command is — produces no derivative spike. And the integral only accumulates when that would not push an already-saturated output further into saturation. |

| | |
|---|---|
| `balance_controller.ino` the 200 Hz tick | Timing uses unsigned subtraction ( `now - last < period` ), which stays correct across the ~71-minute `micros()` rollover. Comparing `now > deadline` does not. On overrun it resyncs rather than firing a burst of catch-up ticks with tiny `dt` . |
| `preprocess.cu` the early `return` | Threads outside the output region must still help load the shared tile, so the bounds check comes *after* `__syncthreads()` . Moving it earlier deadlocks the block. |
| `main.cpp` `FrameGrabber` | `cv::VideoCapture` buffers frames internally. Reading on the pipeline's thread means that whenever the pipeline is briefly slow, latency grows *without bound* and no amount of GPU optimisation helps — the delay is in a queue, not in compute. The grabber keeps exactly one frame: the newest. |

## 1.5 "Where does X happen?"

| Question | Answer |
|---|---|
| Where is the image resized? | `preprocess.cu` , both kernels; geometry from `preprocess_math.cuh` |
| Where do boxes become camera pixels? | `postprocess.cpp` → `decode_yolov8()` , via `dst_to_src_x/y()` |
| Where is the target chosen among several? | `postprocess.cpp` → `select_target()` (confidence minus a distance penalty) |
| Where does jitter get removed? | `command_shaper.cpp` → deadband, then low-pass, then slew limit |
| Where is the robot told to stop? | Three places — see the degradation table in Part 0 |
| Where is the model input size decided? | Read from the engine: `TrtEngine::input_w()/input_h()` . Never hard-coded. |
| Where is the output tensor layout decided? | `main.cpp` → `infer_layout()` , inferred from the engine's dims |
| Where is the angle computed from the IMU? | `balance_controller.ino` , step 3 of `loop()` : `atan2f(ax, az)` , then the filter |
| Where is the fall detected? | `balance_core.h` → `SafetyMonitor` |
| Where do motors actually get written? | `balance_controller.ino` → `motors_drive()` / `motors_coast()` |
| Where is the serial line parsed? | `balance_core.h` → `CommandParser` (byte-at-a-time, never blocks) |
| Where are the latency numbers produced? | `stage_profiler.cuh` (GPU) and `host_profiler.h` (CPU/network); printed on exit |

PART 2

# Bringing it up in the real world

NON-NEGOTIABLE RULES

1. **Never tune with the wheels on the ground and your hands away.** Hold the robot, or prop it so the wheels just clear the surface. A badly tuned balance loop drives at full PWM into whatever is nearest.
2. **Confirm the angle sign before arming the motors once.** With the sign inverted the robot drives itself over, smoothly and confidently, and it looks exactly like a tuning problem. Stage 1 exists only for this.
3. **Charge LiPo cells on a balance charger, never unattended, never in the robot.** This is a fire risk, not a performance note.
4. **Power the ESP32-CAM from the UBEC, not from the Mega's 5 V pin.** The camera's inrush browns out the Mega's regulator and presents as `Camera init failed 0x105` — a power fault that reads like a software fault.
5. **The level shifter on Mega TX → ESP32 RX is mandatory.** 5 V into a 3.3 V pin damages the ESP32 — sometimes immediately, sometimes next week, which is worse.
6. **Keep a hand on the battery connector during first full runs.** It is the fastest reliable off switch you have.

## 2.1 Seven stages, each with a gate

Do not proceed past a gate that has not passed. Every stage is testable without the stages after it — that is why the helper sketches and the `--no-send` flag exist.

STAGE 0 **Tests, on a laptop, with no hardware at all**

```
make -C test
```

Four suites: letterbox geometry, detector decoding, command shaping, and the entire balance-loop logic including a closed-loop pendulum simulation. No GPU, no TensorRT, no OpenCV, no network. Takes about a second.

> **Gate:** all four suites report passing. If the pendulum simulation fails, the control logic or its sign convention is broken — fix that before touching hardware.

### STAGE 1 IMU calibration and the angle sign check

Flash `mcu/imu_calibrate.ino`. Open the Serial Monitor at 115200. Follow its prompts: hold still for the bias measurement, then tilt the robot *forward* and watch the printed angle.

- Angle goes **positive** when leaning forward → `IMU_ANGLE_SIGN = +1.0f`
- Angle goes negative → `IMU_ANGLE_SIGN = -1.0f`
- Rock it forward quickly: `rate` must have the same sign as the angle's direction of travel, otherwise flip `IMU_RATE_SIGN`

Also note the angle when the robot is balanced upright by hand — that is `ANGLE_TRIM_DEG`. And check which accelerometer axis reads ±1.0 at rest; the code assumes Z is vertical and X points along the direction of travel.

> **Gate:** you can state both signs with confidence, and the angle tracks your hand smoothly without jumping.

### STAGE 2 Motors, wheels off the ground

Flash `balance_controller.ino` with the signs from Stage 1. Prop the robot so the wheels spin freely. Power on — **the motors will not move**, by design: `SafetyMonitor` starts disarmed and requires the robot to be held within 4° of upright for 0.75 s before it arms. Watch `arm%` climb on the USB debug line.

Once armed, tilt gently forward. The wheels must spin *forward*. If one wheel is backwards, flip that wheel's `LEFT_INVERT` / `RIGHT_INVERT`. If both are backwards, your angle sign is wrong — go back to Stage 1. **Never fix a direction problem by inverting the PID output**; that also inverts the balance response.

> **Gate:** arming works, both wheels respond in the correcting direction, and `ovr=` is not climbing (that would mean the 5 ms loop is missing its deadline).

### STAGE 3 Tune the balance loop

Still propped, hands ready. One gain at a time. Full procedure is in §3.1 and at the bottom of `balance_controller.ino`.

> **Gate:** the robot holds itself upright unassisted for 10+ seconds and recovers from a gentle push. **No vision hardware is connected yet.**

### STAGE 4 ESP32-CAM alone, nothing attached

Set WiFi credentials and the static IP, flash, read the IP off the serial monitor. Then from the laptop:

```
curl "http://<ip>/status"                      # uptime, RSSI, counters
curl "http://<ip>/cmd?lean=0.00&turn=0.00"     # expect: OK
```

Open `http://<ip>/` in a browser — you should see live video. Check `RSSI` on `/status`: weaker than about −70 dBm will stutter.

> **Gate:** video in a browser, `/cmd` returns OK, RSSI acceptable from wherever the robot will actually be driven.

### STAGE 5 The UART hop, with the motors unable to move

Wire the UART *including the divider*. Flash `mcu/uart_echo_test.ino` to the Mega — it never touches the motors, so the robot can lie on its side. Open the Serial Monitor, then from the laptop:

```
curl "http://<ip>/cmd?lean=1.50&turn=-0.25"   # line appears, marked -> OK
curl "http://<ip>/telemetry"                  # sweeping angle, AGE under ~100
```

Nothing arriving → wrong pins, no common ground, or the divider is backwards. Garbage characters → baud mismatch.

> **Gate:** commands arrive and parse, telemetry flows back and is fresh. Both directions proven before the balance firmware returns.

### STAGE 6 The vision pipeline, transmitting nothing

```
./build/verify_preprocess --bench          # kernel vs OpenCV, and timings
./build/reflex_cv --stream http://<ip>:81/stream \
                  --engine models/model.engine --no-send
```

`--no-send` shapes, displays and counts commands but never transmits them. Walk in front of the camera: the box should track you, the turn bar should lean the way you moved, and the values should move *smoothly*. This is where you tune the tracking gains (§3.2), with zero risk.

> **Gate:** `verify_preprocess` reports PASS; detections are stable; commands look sensible and smooth; frame rate and `dropped` count are acceptable.

## STAGE 7 Everything at once

Balance firmware on the Mega, bridge on the ESP32, pipeline on the laptop, `--no-send` removed. Floor, space, hand near the battery.

Expect to spend real time here. The thing that will need adjusting is **how aggressive the vision commands are**, against a real PID loop and real WiFi latency rather than against printed numbers. Adjust `ShaperConfig` on the laptop. **Do not retune the balance PID** — if it needs retuning once vision is attached, the commands are too aggressive, which is a laptop-side problem.

> **Gate:** it tracks and follows without falling, and recovers sensibly when the target leaves the frame.

PART 3

# Tuning, measuring, and what breaks

## 3.1 Tuning the balance loop

Wheels just clear of the ground, hands ready, one gain at a time. Starting values are already in the file and are the ones the closed-loop simulation stabilises with for a 12 cm centre of mass — a real starting point, not a guess, but every chassis differs.

| Step | Change | What to look for |
|---|---|---|
| 1 | `KI = 0`, `KD = 0`, raise `KP` | Raise until the robot *oscillates* steadily — not until it balances. Then back off about 30%. |
| 2 | raise `KD` | Until the oscillation damps and it feels stiff against a push. Too much makes it buzz at the gearbox's resonance — if it sounds angry, back off. |
| 3 | add `KI`, small | Until the slow one-directional drift stops. Too much gives a slow wallowing oscillation over a second or two — much slower than the KP oscillation, which is how you tell them apart. |
| 4 | `ANGLE_TRIM_DEG` | Kills any remaining consistent creep. If it drifts forward, nudge positive. |
| 5 | `MOTOR_DEADZONE` | Ramp one wheel off the ground and note the PWM where it turns reliably. Too high → jittery; too low → small corrections do nothing. |

> **IF IT DRIFTS ACROSS THE ROOM BUT NEVER FALLS**
>
> That is not a tuning failure. Angle-only balancing cannot distinguish "standing still" from "rolling at constant speed" — both are perfectly upright. The fix is wheel-velocity feedback: set `ENABLE_ENCODER_DAMPING 1` and tune `VEL_KD` upward until it holds position. Do this only after the angle loop is tuned.

## 3.2 Tuning the tracking behaviour

All on the laptop, in `ShaperConfig` (`src/command_shaper.h`). Tune these with `--no-send` before ever tuning them against a moving robot.

| Symptom | Change | Note |
|---|---|---|
| Twitches at a stationary target | raise `turn_deadband` | The deadband eases in rather than stepping, so raising it costs no range |
| Follows sluggishly, lags behind | lower `smooth_tau_s` | It is a time constant, so behaviour stays the same as frame rate varies |
| Jerks when the target moves fast | lower `max_turn_rate` | Bounds how fast even a genuine change reaches the motors |
| Turns too hard and loses balance | lower `turn_gain`, or `TURN_AUTHORITY` on the Mega | A balancing robot that turns hard loses the wheel authority it needs to stay upright |

| | | |
|---|---|---|
| Flips between two people | raise `stickiness` ( `PostprocConfig` ) | Penalises candidates far from last frame's target |
| Stops too close / too far | `desired_height_frac` | Bounding-box height is the distance proxy — crude but monotonic and needs no calibration |
| Keeps turning after target leaves | lower `hold_s` | Below ~0.2 s it will stutter on single missed detections |

## 3.3 Filling in the latency table

`reflex_cv` prints both timing tables on exit, plus markdown rows ready to paste into the README.

```
./build/reflex_cv --stream http://<ip>:81/stream --engine models/model.engine \
                  --no-send --headless --max-frames 1000 --csv run1.csv
```

Read **p95, not max**. Max is usually one startup outlier; p95 is what the robot actually lives with frame after frame.

| To measure | How |
|---|---|
| Shared memory, before/after | `verify_preprocess --bench` times both kernel variants back to back. That is the honest before/after for the optimisation claim. |
| FP16, before/after | Build a second engine without `--fp16` , run both with `--max-frames` , compare the inference row. |
| Batching's real cost | `--batch 1` vs `--batch 4` . Throughput rises, end-to-end latency rises too — for a single camera driving a balance loop, latency is what matters. |
| The network's contribution | Compare the capture-wait row across resolutions using `/res?size=` . Restart the pipeline after changing it. |
| Cost of profiling itself | `--no-profile` . Each stage's `stop()` synchronises, so measurement is not free. |

## 3.4 Failure modes

### Vision pipeline

| Symptom | Cause and fix |
|---|---|
| `deserializeCudaEngine failed` | The engine was built on a different GPU, TensorRT version or driver. Engines are not portable. Rebuild locally with `build_engine.sh` . |
| `cannot open stream` , but the browser shows video | OpenCV built without FFMPEG. Reinstall an OpenCV with FFMPEG support. |
| Boxes consistently offset from objects | The letterbox mapping disagrees between preprocess and postprocess. Run `test_preprocess_math` ; it checks the roundtrip exactly. |
| Detections are nonsense but the model is fine | Preprocessing. Run `verify_preprocess` — its failure output names the likely cause from the error's spatial pattern. |
| Latency climbs steadily the longer it runs | Frames queueing somewhere. Check `dropped` is rising (healthy — the grabber is discarding stale frames) rather than flat. |

| | |
|---|---|
| Engine input rejected as fp16 | Rebuilt with `--inputIOFormats=fp16:chw`. Use plain `--fp16`: it runs the network in half precision while keeping I/O tensors fp32. |
| Command values with commas as decimal points | A locale set `LC_NUMERIC`. Do not call `setlocale`; the ESP32's `atof` would read `1,25` as `1`. |

### Link and bridge

| Symptom | Cause and fix |
|---|---|
| `Camera init failed 0x105` | Power, not software. Feed 5 V from the UBEC, not the Mega. Reseat the camera ribbon. |
| Command latency spikes of 100–200 ms | `WiFi.setSleep(false)` missing or removed. Modem sleep is on by default and is the worst default for this application. |
| Stream works, `/cmd` never answers | Both on one port. They must be separate server instances — see decision ⑦. |
| Stream stutters | RSSI. Weaker than −70 dBm stutters. The antenna is a PCB trace next to the battery and motor wiring; moving the board a few centimetres helps more than you would expect. |
| Mega sees nothing on the UART | In order: common ground, GPIO14 → Mega pin 19, both at 115200, divider on the other direction. Use `uart_echo_test.ino`. |
| ESP32 reboots repeatedly once the motors run | Motors browning out the shared 5 V rail. That is what the separate UBEC feed is for. |

### Balance loop

| Symptom | Cause and fix |
|---|---|
| Drives itself over, smoothly | Angle sign inverted. Stage 1. Not a tuning problem. |
| Motors never move | Working as designed — hold it upright to arm. Watch `arm%` on USB. |
| Lurches violently when set down | Integral wound up while held. Already guarded by the anti-windup and by resetting the integrator while disarmed — if it still happens, check the robot really is reaching the disarmed state. |
| Buzzes and twitches constantly | `KD` too high, or `MOTOR_DEADZONE` too high. Also try `QUIET_PWM 1`. |
| Slow wallowing oscillation | `KI` too high. Distinguishable from a KP oscillation by its much longer period. |
| Drifts across the room, never falls | Expected for angle-only control. Enable encoder damping — see the callout in §3.1. |
| `ovr=` climbing | The 200 Hz loop is missing its deadline. Usually I2C left at 100 kHz instead of 400 kHz, or debug printing moved into the control path. |
| Angle jumps erratically | Byte-swapped I2C reads, a loose SDA/SCL wire, or missing pull-ups. See `wire_read16()` in §1.4. |

## 3.5 Demo day

- **Decide your fallback before you run out of time.** A robot that balances perfectly and does one simple robust thing beats a fragile impressive one. The simplest credible fallback: follow a brightly coloured ball with `--class 32`, which is far easier than tracking people in a crowded room.

- **Test the room's WiFi in advance**, and bring a phone hotspot as a tested backup. A demo room full of laptops is the worst 2.4 GHz environment you will meet.
- **Re-run `imu_calibrate` at the venue.** Gyro bias drifts with temperature, and the firmware re-measures at boot — but confirm the robot really was still during that three-second window.
- **Have the latency table filled in.** The measured before/after numbers are the most interesting thing about the project to anyone technical, and they take one headless run to produce.
- **Charge batteries the night before**, on a balance charger, attended.

PART 4

# Verification status — what is actually proven

This matters more than any other section. The codebase was written and checked in a cloud container with **no GPU, no TensorRT SDK, no ESP32 toolchain and no robot**. Here is exactly what that means, per file.

TESTED compiled and executed, with assertions COMPILED built and linked, never executed
TYPE-CHECKED against a stub, not the real SDK UNBUILT no compiler has seen it

| Component | Status | Evidence |
|---|---|---|
| `test/* (4 suites)` | TESTED | 6,667,276 assertions, all passing, via both `make` and `ctest` |
| `mcu/balance_core.h` | TESTED | 164 checks incl. a closed-loop pendulum sim: balances from 3°, recovers from a 2.5 rad/s shove, holds a commanded 2° lean |
| `src/preprocess_math.cuh` | TESTED | 6,666,415 checks; mapping roundtrip exact; tile window proven to cover every bilinear tap at six resolutions |
| `src/postprocess.*` | TESTED | 68 checks: decode against hand-computed coordinates, both tensor layouts, NMS, sticky selection |
| `src/command_shaper.*` | TESTED | 629 checks; detector jitter reduced to 19% of raw amplitude; loss-decay monotonic; large-`dt` step bounded |
| `src/preprocess.cu` | COMPILED | nvcc 12.0 → sm_86 and sm_89, no warnings. **Never executed — there is no GPU here.** |
| `whole vision pipeline` | COMPILED | Links into a working binary against real OpenCV 4.6 headers; `--help` runs |
| `tools/verify_preprocess.cpp` | COMPILED | Compiles and links |
| `mcu/balance_controller.ino` | COMPILED | avr-g++ 7.3 for atmega2560, links: 18,348 B flash (7.2%), 747 B RAM (9.1%) |
| `mcu/imu_calibrate.ino` | COMPILED | 11,830 B flash, 424 B RAM |
| `mcu/uart_echo_test.ino` | COMPILED | 9,610 B flash, 429 B RAM |
| `src/trt_infer.*, src/main.cpp` | TYPE-CHECKED | Against `test/stubs/NvInfer.h`, a hand-written subset. **If a signature in that stub is wrong, this passes and the real build fails.** |
| `esp32cam/…uart_bridge.ino` | UNBUILT | The ESP32 toolchain could not be fetched here. Written against the stable ESP32 Arduino core and `esp_http_server` APIs, but **no compiler has seen this file.** |

**WHAT TO EXPECT ON FIRST CONTACT WITH HARDWARE**

- **The ESP32 sketch will probably need small fixes on its first build.** It is the one file with no compiler verification. Budget an hour; expect an include or an enum name, not a design problem.
- **The TensorRT wrapper may need signature corrections** against your actual SDK version. The error messages will be compiler errors at the call sites, which are few and clearly marked. The `#error` at the top of `trt_infer.h` catches a too-old SDK immediately.
- **The CUDA kernel has never run.** Its arithmetic is heavily unit-tested on the host and it compiles for both target architectures, but the first thing to do after building is `verify_preprocess`, which exists precisely to check it against OpenCV on a real GPU.
- **The PID gains are simulation-derived**, for a 12 cm centre of mass. They are a real starting point, not a guess — but your chassis is not the simulation, and you will retune.

APPENDIX A

# Command-line reference

## `reflex_cv` — the vision pipeline

| Flag | Default | Meaning |
|---|---|---|
| `--stream URL` | `192.168.4.1` | MJPEG stream. Note port **81**. |
| `--engine PATH` | `models/model.engine` | Engine built on *this* machine |
| `--esp32 HOST[:PORT]` | stream's host, :80 | Command endpoint; usually inferred |
| `--class N` | `0` | COCO class: 0 person, 32 sports ball |
| `--conf F / --iou F` | `0.35 / 0.45` | Detection and NMS thresholds |
| `--batch N` | `1` | Frames per inference. Raises throughput, raises latency. |
| `--send-hz F` | `20` | Max command rate to the robot |
| `--fps-limit F` | unlimited | Cap pipeline rate to save GPU |
| `--max-frames N` | until Ctrl-C | For repeatable benchmarks |
| `--csv PATH` | off | Per-frame stage timings on exit |
| `--headless` | off | No display window |
| `--no-send` | off | **Use first.** Shapes and prints commands, transmits nothing. |
| `--no-profile` | off | Skip CUDA-event timing (each stage's stop synchronises) |
| `--verbose` | off | TensorRT info-level logging |

## Everything else

```
make -C test                      # four host suites, no hardware needed
make -C test stub-check           # type-check the TensorRT files without the SDK
ctest                            # same suites, from the CMake build dir

cd models
python3 export_onnx.py            # YOLOv8 -> model.onnx (commit this)
./build_engine.sh model.onnx model.engine     # per machine (never commit)

./build/verify_preprocess --bench            # kernel vs OpenCV, plus timings

curl "http://<esp32>/status"                 # bridge diagnostics
curl "http://<esp32>/cmd?lean=0&turn=0"      # stop steering
curl "http://<esp32>/telemetry"              # last line from the Mega
curl "http://<esp32>/res?size=svga"          # change resolution, then restart
```

APPENDIX B

# Every constant you will want to change

## Mega — `mcu/balance_controller.ino`

| Constant | Default | Change it when |
|---|---|---|
| `IMU_ANGLE_SIGN` | +1.0 | Stage 1 says so. **Get this right first.** |
| `IMU_RATE_SIGN` | +1.0 | Stage 1 says so |
| `ANGLE_TRIM_DEG` | 0.0 | It creeps consistently one way |
| `PID_KP / KI / KD` | 18 / 60 / 1.2 | Stage 3, in that order |
| `FILTER_TAU_S` | 0.75 | Angle noisy (raise) or laggy (lower) |
| `MOTOR_DEADZONE` | 35 | Measured on your motors |
| `TURN_AUTHORITY` | 60 | Turns too hard / barely turns |
| `FALL_ANGLE_DEG` | 35 | Rarely |
| `ARM_ANGLE_DEG / DWELL` | 4 / 0.75 | Arming feels fussy |
| `CMD_TIMEOUT_MS` | 400 | Rarely — this is a safety limit |
| `LEFT/RIGHT_INVERT` | false | A wheel spins backwards |
| `QUIET_PWM` | 0 | After it balances, if the whine annoys you |
| `ENABLE_ENCODER_DAMPING` | 0 | After it balances, to stop position drift |

## Laptop — `src/command_shaper.h` and `src/postprocess.h`

| Constant | Default | Change it when |
|---|---|---|
| `turn_gain` | 0.9 | Turns too hard or too weakly |
| `turn_deadband` | 0.05 | Twitches at a still target |
| `lean_gain` | 12.0 | Approaches too fast or too slowly |
| `desired_height_frac` | 0.45 | Stops too close or too far |
| `max_lean_deg` | 4.0 | Rarely — this bounds how hard it drives |
| `smooth_tau_s` | 0.12 | Sluggish (lower) or jittery (raise) |
| `max_turn_rate` | 4.0 | Jerks on fast target motion |
| `hold_s / decay_s` | 0.35 / 0.40 | Keeps turning after the target is gone |
| `stickiness` | 0.5 | Flips between two candidates |

## ESP32-CAM — the top of the sketch

| Constant | Default | Change it when |
|---|---|---|
| `WIFI_SSID / WIFI_PASS` | — | Immediately. 2.4 GHz only. |
| `USE_STATIC_IP / STATIC_IP` | 1 / .1.50 | To match your network. Worth keeping on. |
| `START_FRAMESIZE` | VGA | Latency experiments; or `/res` at runtime |
| `START_QUALITY` | 12 | Lower = better image, bigger frames |
| `UART_RX_PIN / TX_PIN` | 13 / 14 | Only if you need the SD card |